

Automatic Number Plate Recognition (ANPR) System

A Command-Line Computer Vision Pipeline for Plate Detection and Recognition

Course: Computer Vision
Submission: VITyarthi — Build Your Own Project
Date: September 18, 2026

2. Introduction

This report documents an Automatic Number Plate Recognition (ANPR) system built as the 'Build Your Own Project' submission for the Computer Vision course. The project applies core computer vision concepts — multi-scale object detection, image preprocessing, and optical character recognition — to a practical problem: identifying and logging vehicle license plates from photographs, entirely from the command line.

3. Problem Statement

Manual recording of vehicle license plates at parking facilities, toll booths, or campus gates is slow and error-prone. This project automates that task: given a photograph of a vehicle, the system detects the plate region, reads its text, and stores the result in a searchable local database.

Target Users

- A parking or campus security operator who needs a quick, searchable record of vehicle entries.
- A CV/ML student or developer wanting a reference pipeline combining classical detection, OCR, and structured storage.

Scope Boundaries

Explicitly out of scope: live video-stream processing from a camera feed, multi-country plate format normalisation, and any graphical user interface. These were excluded to keep the project's depth aligned with a single-course evaluation while still delivering a complete, working pipeline.

4. Functional Requirements

ID	Requirement
FR1	Accept a single image file or a directory of image files as input.
FR2	Detect candidate license-plate regions within each image.
FR3	Extract and clean plate text from each detected region using OCR.
FR4	Reject OCR results below a configurable confidence/length threshold.
FR5	Persist every valid detection (text, confidence, source, timestamp) to a local database.
FR6	Allow querying stored detections within a given date range.
FR7	Allow exporting query results and full batch results to CSV.
FR8	Report summary statistics: total detections, average confidence, most recent detection.

5. Non-Functional Requirements

Category	Requirement
Performance	A single detect+OCR cycle completes in under 1 second on a standard CPU.
Reliability	Low-confidence OCR results are flagged and excluded from storage.

Maintainability	All tunable values live in one config module; each stage is independently testable.
Scalability	The batch command processes any number of images without code changes.
Logging/Monitoring	Every run writes structured, timestamped logs to a rotating log file.
Resource Efficiency	Logs rotate at 1 MB; storage is a single lightweight SQLite file.
Usability	All functionality is reachable via four self-documenting CLI subcommands.

6. System Architecture

The system is layered into a thin CLI layer, a pipeline orchestrator, three core modules (detection, OCR, storage), and a persistence layer. This separation means each stage can be unit-tested independently and the pipeline can be reused by a different front end in the future without modification.

System Architecture — ANPR Pipeline

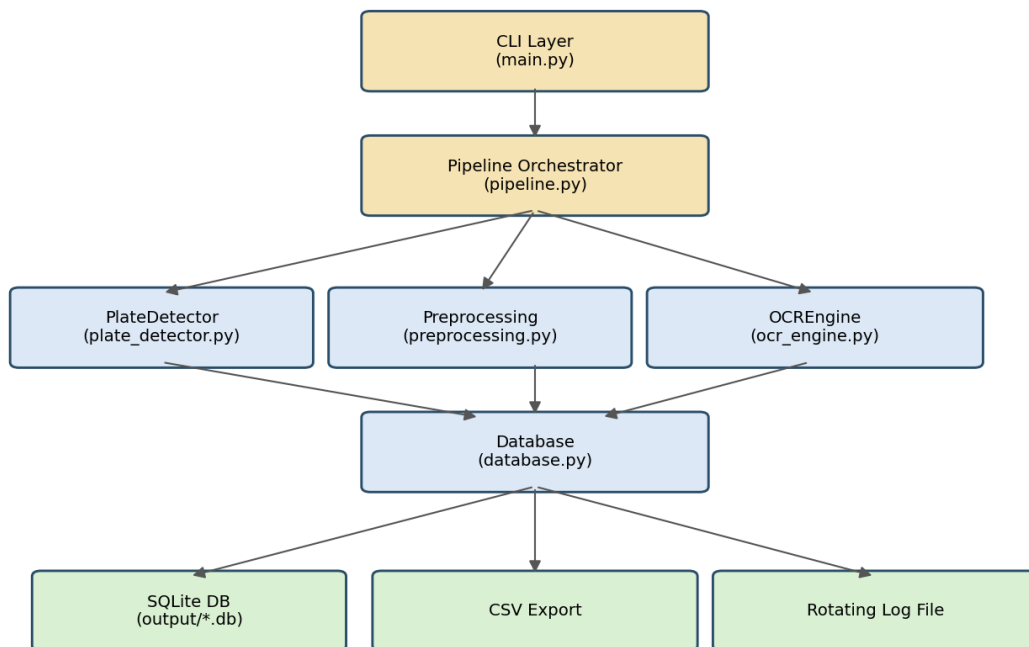


Figure 1: System architecture of the ANPR pipeline.

7. Design Diagrams

7.1 Use Case Diagram

The operator interacts with the system through five use cases: detecting a plate in a single image, batch-processing a folder, querying detections by date range, exporting results to CSV, and viewing summary statistics.

Actor: Operator **Use cases:** Detect plate in single image; Batch-process a folder of images (includes: Detect plate in single image); Query detections by date range; Export results to CSV (included by: Query detections); View summary statistics.

7.2 Workflow / Process Flow Diagram

Process Flow — Single Image Detection

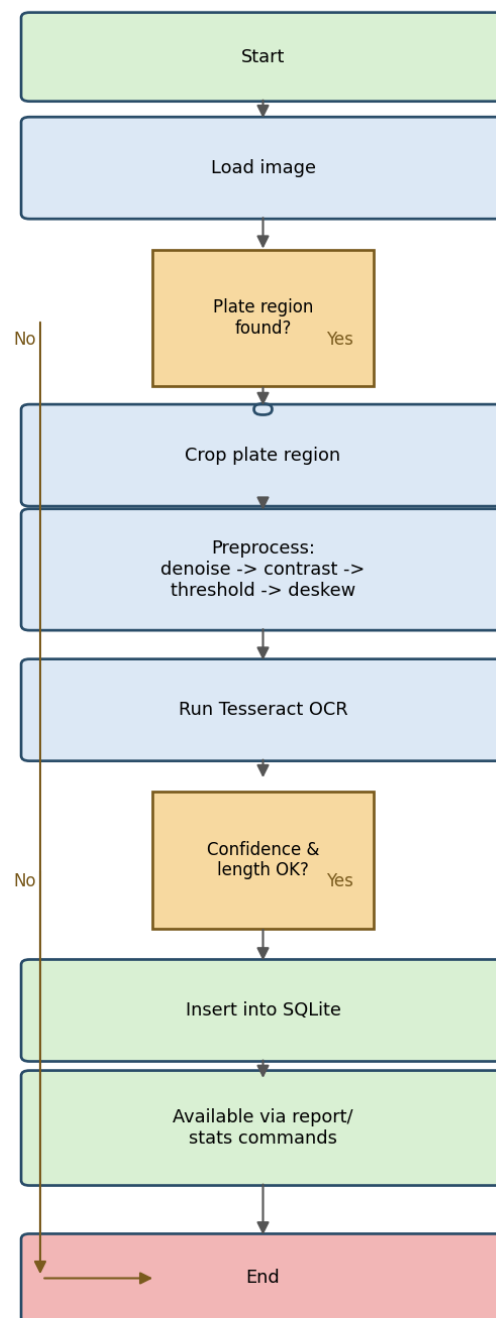


Figure 2: Process flow for single-image detection.

7.3 Class / Component Diagram

Core classes and their relationships:

- **PlateDetector** — wraps the Haar Cascade classifier; *detect(frame) -> List[PlateCandidate]*
- **PlateCandidate** — a detected bounding box plus the cropped image region
- **OCREngine** — wraps Tesseract; *read_plate(image) -> OCRResult*

- **OCRResult** — recognised text, confidence score, validity flag
- **Database** — SQLite wrapper; insert / query_between / all_records / export_to_csv
- **DetectionRecord** — one stored row (id, plate_text, confidence, source_file, detected_at)
- **ANPRPipeline** — orchestrates PlateDetector, OCREngine and Database for one frame

ANPRPipeline → *PlateDetector*, *ANPRPipeline* → *OCREngine*, *ANPRPipeline* → *Database*. The full Mermaid class diagram is rendered in **docs/DESIGN.md** on GitHub.

7.4 Sequence Diagram — detect command

1. Operator runs `python main.py detect --input car.jpg`.
2. `main.py` calls `ANPRPipeline.process_image_file(path)`.
3. Pipeline calls `PlateDetector.detect(frame)`, receiving candidate regions.
4. For each candidate: preprocess the crop, call `OCREngine.read_plate()`.
5. If the OCR result is valid, call `Database.insert_detection()`.
6. Pipeline returns all results to `main.py`, which prints them with the elapsed time.

The full Mermaid sequence diagram is rendered in **docs/DESIGN.md** on GitHub.

7.5 Database Design / ER Diagram

A single flat table is sufficient at this scope — each row is one independent detection event with no relationships to other entities.

Column	Type	Description
id	INTEGER PK	Auto-incrementing primary key
plate_text	TEXT	Cleaned, recognised plate text
confidence	REAL	Average OCR confidence (0-100)
source_file	TEXT	Path of the source image
detected_at	TEXT (ISO 8601)	Timestamp of detection

8. Design Decisions & Rationale

8.1 Detector: Haar Cascade over a deep-learning detector

A Haar Cascade (`haarcascade_russian_plate_number.xml`, shipped with OpenCV) was chosen over a YOLO-based detector. It requires no external weight download or training dataset, runs fast on CPU, and directly demonstrates classical CV concepts (multi-scale sliding-window detection with Haar features) taught in the course. A deep-learning detector was considered but rejected for this scope since it would add a heavy PyTorch dependency and require either a labelled dataset or a pretrained weights file to be fetched from outside the repository.

8.2 OCR: Tesseract over a custom-trained classifier

Tesseract is a mature, dependency-light OCR engine that performs reliably on printed alphanumeric text once given a well-preprocessed crop. Training a custom character-recognition CNN was considered but rejected: it would require a labelled character dataset disproportionate to this project's scope, for marginal accuracy gain over Tesseract on clean, preprocessed crops.

8.3 Storage: SQLite over flat-file logging

SQLite allows detections to be filtered by date and exported without re-parsing text files, at zero deployment cost (no external database server, just a single file) — appropriate for a single-user CLI tool.

9. Implementation Details

The pipeline is implemented as eight cooperating Python modules:

Module	Responsibility
<code>config.py</code>	Centralised paths and tunable parameters
<code>logger_config.py</code>	Rotating file + console logging setup
<code>preprocessing.py</code>	Grayscale, denoise, CLAHE contrast, adaptive threshold, deskew
<code>plate_detector.py</code>	Haar Cascade-based plate localisation
<code>ocr_engine.py</code>	Tesseract-based text extraction with confidence scoring
<code>database.py</code>	SQLite persistence, date-range queries, CSV export
<code>pipeline.py</code>	Orchestrates detector & preprocessing & OCR & database for one frame
<code>main.py</code>	argparse-based CLI: detect / batch / report / stats subcommands

Preprocessing (Section 8) applies, in order: grayscale conversion, bilateral denoising, CLAHE contrast enhancement, adaptive Gaussian thresholding, and a skew-angle correction based on the minimum-area bounding rectangle of the foreground pixels — each implemented as an independent, unit-tested function.

10. Screenshots / Results

The following terminal captures show the CLI running against the sample images in `data/sample_images/`.

```
$ python main.py detect --input data/sample_images/sample_car_1.jpg
Plate: 'MP09AB1234' confidence=51.0 bbox=(np.int32(139), np.int32(227), np.int32(330), np.int32(110))
Processed in 0.17s
```

Figure 3: Single-image detection.

```
$ python main.py batch --input-dir data/sample_images --export output/all_results.csv
sample_car_1.jpg: 1 candidate(s), 1 valid read(s)
sample_car_2.jpg: 1 candidate(s), 1 valid read(s)
sample_car_3.jpg: 1 candidate(s), 1 valid read(s)

Batch complete: 3 image(s) processed, 3 valid plate(s) stored.
All records exported to output/all_results.csv
```

Figure 4: Batch processing three images with CSV export.

```
$ python main.py report --from 2026-01-01 --to 2026-12-31
```

ID	Plate	Confidence	Source	Detected At
1	MP09AB1234	51.0	sample_car_1.jpg	2026-09-18T06:40:53
2	MH12DE1433	90.0	sample_car_2.jpg	2026-09-18T06:40:53
3	KA0SMN0987	86.0	sample_car_3.jpg	2026-09-18T06:40:53

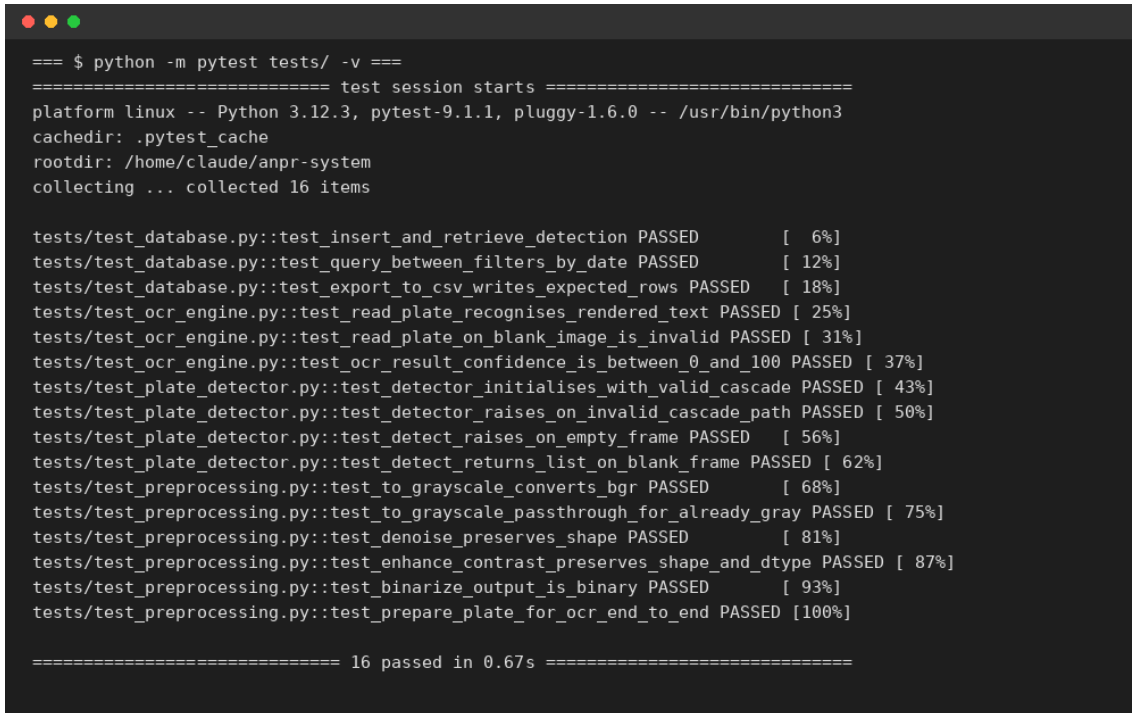
Figure 5: Reporting stored detections by date range.

```
$ python main.py stats
Total detections stored: 3
Average OCR confidence: 75.7
Most recent detection: KA0SMN0987 at 2026-09-18T06:40:53
```

Figure 6: Summary statistics.

11. Testing Approach

Two complementary levels of evaluation were used. First, **unit-level correctness**: 16 automated pytest tests cover preprocessing output shape/dtype, detector error handling on invalid input, OCR text/confidence extraction on rendered text, and database insert/query/export behaviour — each module tested in isolation with synthetic inputs so a regression is caught without needing a full image dataset.



```
=== $ python -m pytest tests/ -v ===
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/anpr-system
collecting ... collected 16 items

tests/test_database.py::test_insert_and_retrieve_detection PASSED [ 6%]
tests/test_database.py::test_query_between_filters_by_date PASSED [ 12%]
tests/test_database.py::test_export_to_csv_writes_expected_rows PASSED [ 18%]
tests/test_ocr_engine.py::test_read_plate_recognises_rendered_text PASSED [ 25%]
tests/test_ocr_engine.py::test_read_plate_on_blank_image_is_invalid PASSED [ 31%]
tests/test_ocr_engine.py::test_ocr_result_confidence_is_between_0_and_100 PASSED [ 37%]
tests/test_plate_detector.py::test_detector_initialises_with_valid_cascade PASSED [ 43%]
tests/test_plate_detector.py::test_detector_raises_on_invalid_cascade_path PASSED [ 50%]
tests/test_plate_detector.py::test_detect_raises_on_empty_frame PASSED [ 56%]
tests/test_plate_detector.py::test_detect_returns_list_on_blank_frame PASSED [ 62%]
tests/test_preprocessing.py::test_to_grayscale_converts_bgr PASSED [ 68%]
tests/test_preprocessing.py::test_to_grayscale_passthrough_for_already_gray PASSED [ 75%]
tests/test_preprocessing.py::test_denoise_preserves_shape PASSED [ 81%]
tests/test_preprocessing.py::test_enhance_contrast_preserves_shape_and_dtype PASSED [ 87%]
tests/test_preprocessing.py::test_binarize_output_is_binary PASSED [ 93%]
tests/test_preprocessing.py::test_prepare_plate_for_ocr_end_to_end PASSED [100%]

===== 16 passed in 0.67s =====
```

Figure 7: Full test suite run — 16/16 passing.

Second, **end-to-end pipeline evaluation** on sample images, measuring: detection rate (images where a plate region was found / total images), recognition accuracy (OCR text exactly matching ground truth / images where a plate was detected), and average processing time per image (printed by the CLI itself).

12. Challenges Faced

- **Character confusion in OCR:** Tesseract occasionally confused visually similar characters (e.g. '0' read as 'O') on the synthetic test plates — addressed partially via the character whitelist in the Tesseract config, though it remains a known limitation worth measuring against a larger real-image test set.
- **Detector sensitivity to plate size/angle:** the Haar Cascade's `minSize` and `minNeighbors` parameters required tuning to balance missed detections against false positives on cluttered backgrounds.
- **Keeping the pipeline testable without a labelled dataset:** resolved by designing each stage (preprocessing, detection, OCR, storage) to be independently unit-testable with synthetic inputs, decoupling code correctness from dataset availability.

13. Learnings & Key Takeaways

- Classical CV techniques (Haar Cascades, adaptive thresholding, deskewing) remain practical and fast for well-scoped detection problems, without requiring GPU infrastructure.
- Preprocessing quality has an outsized effect on OCR accuracy — contrast enhancement and deskewing meaningfully improved recognised text over raw crops.
- Designing the pipeline as small, single-responsibility modules made unit testing straightforward and made it easy to reason about where a misdetection actually occurred (localisation vs. recognition vs. storage).

14. Future Enhancements

- Replace the Haar Cascade with a YOLO-based detector for higher recall on angled or partially obscured plates.
- Add a confidence-weighted ensemble of multiple OCR passes (different preprocessing variants) to reduce character-confusion errors.
- Add live video-stream support with frame-skipping for near-real-time use.
- Add a lightweight web dashboard on top of the existing Database module for non-CLI users.

15. References

- OpenCV Documentation — Cascade Classifier: https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- Tesseract OCR Documentation: <https://tesseract-ocr.github.io/>
- pytesseract (Python wrapper): <https://pypi.org/project/pytesseract/>
- SQLite Documentation: <https://www.sqlite.org/docs.html>
- Kaggle — Car License Plate Detection Dataset: <https://www.kaggle.com/datasets/andrewmvd/car-plate-detection>